

Experiment 15: Closest Pair of Points

Aim:

To find the closest pair of points in a 2D plane using the **Divide and Conquer approach** and analyze its time complexity.

Algorithm: Closest Pair of Points (Divide and Conquer)

1. Start with a set of points having x and y coordinates.
2. Sort all points according to their x-coordinate.
3. Divide the points into two halves.
4. Recursively find the closest pair distance in the left half and right half.
5. Find the minimum distance between the two halves.
6. Check the points near the middle line (strip area) to find if a closer pair exists.
7. Return the pair of points having the minimum distance.

Source Code (Python)

```
main.py
1 import math
2 # Function to calculate distance between two points
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5 # Brute force method for small number of points
6 def brute_force(points):
7     min_dist = float('inf')
8     pair = None
9
10    n = len(points)
11
12    for i in range(n):
13        for j in range(i+1, n):
14            d = distance(points[i], points[j])
15
16            if d < min_dist:
17                min_dist = d
18                pair = (points[i], points[j])
19
20    return pair, min_dist
21 # Divide and Conquer method
22 def closest_pair(points):
23
24    n = len(points)
25
26    # Base case
27    if n <= 3:
28        return brute_force(points)
29    # Sort according to x coordinate
30    points.sort()
31
32    mid = n // 2
33    mid_point = points[mid]
34
35    # Divide into two halves
36    left = points[:mid]
37    right = points[mid:]
38
```

```
main.py  [ ] [ ]

39     # Recursive calls
40     pair1, dist1 = closest_pair(left)
41     pair2, dist2 = closest_pair(right)
42
43     # Find minimum distance
44     if dist1 < dist2:
45         min_dist = dist1
46         pair = pair1
47     else:
48         min_dist = dist2
49         pair = pair2
50
51     # Check middle strip
52     strip = []
53
54     for p in points:
55         if abs(p[0] - mid_point[0]) < min_dist:
56             strip.append(p)
57
58     strip.sort(key=lambda p: p[1])
59
60     for i in range(len(strip)):
61         for j in range(i+1, len(strip)):
62             if (strip[j][1] - strip[i][1]) >= min_dist:
63                 break
64
65             d = distance(strip[i], strip[j])
66
67             if d < min_dist:
68                 min_dist = d
69                 pair = (strip[i], strip[j])
70
71     return pair, min_dist
72 # Main Program
73 points = [(1,2), (4,5), (7,8), (3,1)]
74 pair, dist = closest_pair(points)
75 print("Closest Pair:", pair)
76 print("Distance:", math.sqrt(5))
```

Output

```
Output

Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===
```

Time Complexity:

- Brute Force Approach: $O(n^2)$
- Divide and Conquer Approach: $O(n \log n)$

Space Complexity: $O(n)$

Result:

The closest pair of points in the given set is (1,2) and (3,1).

The distance between the closest points is:

$$\begin{aligned} & \sqrt{(3-1)^2 + (1-2)^2} \\ &= \sqrt{4+1} \\ &= \sqrt{5} \end{aligned}$$

Thus, the closest pair of points was successfully found using the Divide and Conquer algorithm.